```
user@ansible-control: ~/ansible-project
$ mkdir -p ~/ansible-project && cd ~/ansible-project
$ nano ansible.cfg
# --- ansible.cfg contents ---
[defaults]
inventory      = ./inventory.ini
remote_user    = devops
host_key_checking = False
retry_files_enabled = False
$ export ANSIBLE_CONFIG=~/ansible-project/ansible.cfg
$ export ANSIBLE_INVENTORY=~/ansible-project/inventory.ini
$ export ANSIBLE_HOST_KEY_CHECKING=False
$ echo $ANSIBLE_CONFIG
/home/devops/ansible-project/ansible.cfg
$ ansible-config view
[defaults]
inventory = ./inventory.ini
remote_user = devops
```

Figure 3: Setting up *ansible.cfg* and exporting environment variables

```
user@ansible-control: ~/ansible-project
$ ansible --version
ansible [core 2.17.4]
  config file = /home/devops/ansible-project/ansible.cfg
  python version = 3.12.3 (main, Apr  9 2025, 08:41:22)
  jinja version = 3.1.2
$ cat inventory.ini
[web]
192.168.1.10
192.168.1.11
$ ansible all -m ping
192.168.1.10 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
192.168.1.11 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
# Both managed nodes reachable and responding
```

Figure 4: Verifying the Ansible version and confirming c onnectivity to managed nodes

```
mkdir -p ~/ansible-project && cd ~/ansible-project

# ansible.cfg
[defaults]
inventory      = ./inventory.ini
remote_user    = devops
host_key_checking = False

export ANSIBLE_CONFIG=~/ansible-project/ansible.cfg
export ANSIBLE_INVENTORY=~/ansible-project/inventory.ini
```

Verifying installation

Once installed and configured, the installation should be verified by checking the version, confirming the configuration file in use, and testing connectivity to managed nodes with the built-in ping module (which checks Python availability over SSH, not ICMP).

```
ansible --version
cat inventory.ini
ansible all -m ping
```

A *SUCCESS* response with 'pong' for each host shows that the control node can reach each managed node and that Ansible modules can be executed — the environment is ready for building and executing playbooks.

Case study: Scaling web server deployment at a retail company

A mid-sized e-commerce company ran a load balanced web application that spanned 12 web servers. New releases of the application and new security patches were performed by the manual process of SSH-ing into each of the 12 web servers. This introduced an issue of configuration drift.

- Manual deployments were slow and blocked releases to a single maintenance window per week.
- Configuration drift caused intermittent bugs that only appeared on some servers.
- Onboarding a new server took a full day of manually replicating existing configuration.
- There was no record of exactly what had been changed on each host over time.

The operations team introduced Ansible as a lightweight automation layer. All 12 web servers were added to an inventory grouped under *[webservers]*, and the existing manual steps were translated into a single, version-controlled playbook covering package updates, application code deployment, configuration file templating, and service restarts.

```
# deploy_webservers.yml
- name: Deploy and configure storefront web servers
  hosts: webservers
  become: true
  serial: 3          # rolling update, 3 servers at a time
  tasks:
    - name: Ensure required packages are present
      ansible.builtin.apt:
        name: ['nginx', 'php8.2-fpm']
        state: present

    - name: Deploy latest application release
      ansible.builtin.git:
        repo: 'https://git.example.com/storefront.git'
        dest: /var/www/storefront
        version: '{{ release_tag }}'

    - name: Template nginx site configuration
      ansible.builtin.template:
        src: templates/storefront.conf.j2
        dest: /etc/nginx/sites-available/storefront.conf
      notify: Reload nginx

  handlers:
    - name: Reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

Continued to page...78